

NullOwn — Architecture & Developer Guide

Purpose of this document: This file is the primary reference for AI-assisted development (Claude Code, Antigravity, Copilot, etc.). It captures the full system design, module contracts, data flows, and technology decisions so that any AI coding assistant can contribute accurately without needing to re-derive context from scratch.

Table of Contents

- 1. [Project Overview](#)
- 2. [Problem Statement](#)
- 3. [High-Level Architecture](#)
- 4. [Three-Layer Design](#)
- 5. [Module Breakdown](#)
- 6. [Technology Stack](#)
- 7. [Directory Structure \(Target\)](#)
- 8. [Smart Contract Interfaces](#)
- 9. [ZK Circuit Design](#)
- 10. [Frontend Architecture](#)
- 11. [Data Flow Diagrams](#)
- 12. [Key Cryptographic Primitives](#)
- 13. [Scope & Constraints](#)
- 14. [Development Conventions](#)
- 15. [References](#)

1. Project Overview

NullOwn is a privacy layer for tokenized Real-World Assets (RWA) on EVM-compatible blockchains. It allows investors to:

Capability	Mechanism
Receive RWA tokens privately	Stealth address protocol (ERC-5564 / ERC-6538)
Hold assets invisibly	One-time stealth addresses unlinked from primary wallet
Prove ownership without disclosure	ZK proofs (Circom + SnarkJS, Groth16)
Satisfy regulatory audit	Permissioned compliance module with on-chain audit log

NullOwn operates as a **privacy overlay** — it does not issue or modify existing RWA tokens (ERC-721 / ERC-1155). It is non-custodial; users control their own keys.

Target Networks: Ethereum Sepolia testnet / Polygon Mumbai (testnet only for this phase).

2. Problem Statement

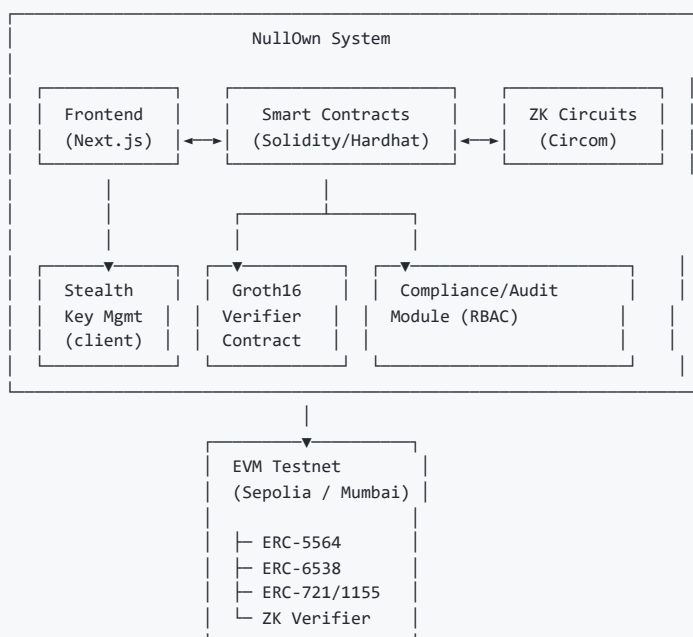
Core Issues

- **Public Ledger Exposure** — All wallet holdings and transaction history are publicly readable. Institutional investors cannot reveal real-time portfolio positions to competitors.
- **The KYC Gap** — KYC confirms identity at onboarding but does nothing to restrict public visibility of holdings post-onboarding.
- **Composability Amplifies Exposure** — Using an RWA token as DeFi collateral links the investor's identity across multiple protocols simultaneously.
- **No On-Chain Proof Without Full Disclosure** — The only way to prove "I own asset Y" is direct wallet inspection, which reveals the entire portfolio.

Design Goal

Enable an investor to receive RWA tokens privately, hold them invisibly, prove ownership when needed, and remain auditable by regulators — all without a single public disclosure.

3. High-Level Architecture



4. Three-Layer Design

Layer 1 — Stealth Address Protocol

Standard: ERC-5564 (stealth transfers) + ERC-6538 (stealth meta-address registry)

Cryptography: Elliptic Curve Diffie-Hellman (ECDH) on secp256k1

How it works

1. **Investor Registration** — The investor registers a `StealthMetaAddress` (spend public key + view public key) on-chain via the ERC-6538 registry contract.
2. **Sender Computes Stealth Address** — The token sender:
 - Generates a random ephemeral key pair (r, R) where $R = r \cdot G$
 - Computes shared secret $S = r \cdot V$ (V = investor's view public key)
 - Derives one-time stealth address $P = \text{hash}(S) \cdot G + K$ (K = spend public key)
3. **On-Chain Announcement** — The sender publishes R (ephemeral public key) to the ERC-5564 `Announcements` contract.
4. **Receiver Scanning** — The investor scans announcements using their private view key v :
 - Computes $S = v \cdot R$
 - Derives expected stealth address and checks against on-chain addresses
5. **Asset Control** — The investor uses their private spend key to sign transactions from the stealth address.

Key Properties

- The public ledger records a transfer to an address with **no linkable history**
 - The investor's primary wallet is **never exposed**
 - Each transfer uses a **fresh one-time address**
-

Layer 2 — Zero-Knowledge Ownership Proofs

Toolchain: Circom 2.x (circuit DSL) + SnarkJS (prover/verifier) + Groth16 proving scheme

Circuit Inputs

Type	Name	Description
Private	tokenId	The specific token ID held
Private	spendKeyHash	Hash of investor's private spend key
Private	merklePathElements[]	Merkle inclusion proof elements
Private	merklePathIndices[]	Left/right flags for Merkle path
Public	merkleRoot	Root of the ownership Merkle tree
Public	threshold	Minimum ownership threshold being proven
Public	nullifier	Prevents double-use of same proof

Circuit Assertions

1. The spendKeyHash correctly derives the stealth address holding tokenId
2. tokenId is included in the Merkle tree (proven via merklePathElements)
3. Ownership quantity $\geq$ threshold
4. nullifier is correctly computed from Poseidon(spendKeyHash, tokenId)

On-Chain Flow

1. Investor runs the prover **client-side** (browser via SnarkJS WASM)
2. Proof + public signals submitted to **Groth16Verifier** smart contract
3. Contract verifies the proof cryptographically via BN254 pairing
4. On success, emits VerifiedOwnership(nullifier, merkleRoot, timestamp) event
5. **No wallet address, token ID, quantity, or identity is disclosed**

Layer 3 — Compliance Bridge

Design: Permissioned audit module with Role-Based Access Control (RBAC)

Roles

Role	Capabilities
DEFAULT_ADMIN_ROLE	Grants/revokes regulator roles
REGULATOR_ROLE	Requests scoped ownership record decryption
Public	No access to ownership records

Audit Flow

1. A pre-authorized regulator (holding `REGULATOR_ROLE`) submits an audit request on-chain
2. The request is **logged immutably** on-chain (transparency of audit activity itself)
3. The compliance module returns encrypted ownership data decryptable only by the audit key
4. The general public and other market participants **never gain access**

Key Design Principle

Compliance auditability and public transparency are **not mutually required**. NullOwn separates them.

5. Module Breakdown

```
NullOwn
├── contracts/
│   ├── stealth/
│   │   ├── ERC5564Announcer.sol      # On-chain ephemeral key announcements
│   │   └── ERC6538Registry.sol       # Stealth meta-address registry
│   ├── zk/
│   │   └── Groth16Verifier.sol        # Auto-generated by SnarkJS (do not hand-edit)
│   ├── compliance/
│   │   └── ComplianceModule.sol      # RBAC-gated audit module
│   └── interfaces/
│       ├── IStealthAnnouncer.sol
│       ├── IStealthRegistry.sol
│       └── IOwnershipVerifier.sol
├── circuits/
│   ├── ownership_proof.circom        # Main ownership proof circuit
│   ├── merkle_inclusion.circom       # Merkle tree membership sub-circuit
│   ├── nullifier.circom             # Nullifier computation sub-circuit
│   └── poseidon/                    # Poseidon hash gadgets
├── scripts/
│   ├── deploy.ts
│   ├── setup-zkeys.ts               # Trusted setup / zkey generation
│   └── seed-registry.ts
├── test/
│   ├── stealth/
│   ├── zk/
│   └── compliance/
├── frontend/
│   ├── app/                         # Next.js 14 App Router pages
│   │   ├── page.tsx                 # Landing / dashboard
│   │   ├── stealth/page.tsx         # Key management & scanning
│   │   ├── prove/page.tsx           # ZK proof generation UI
│   │   └── compliance/page.tsx      # Compliance attestation export
│   ├── components/
│   ├── lib/
│   │   ├── stealth.ts               # ECDH / stealth address logic
│   │   ├── zkProver.ts              # SnarkJS WASM wrapper
│   │   ├── merkle.ts                # Merkle tree construction (Poseidon)
│   │   └── contracts.ts             # Contract ABIs & addresses
│   ├── public/
│   │   └── circuits/                # WASM & zkey files for browser proving
├── hardhat.config.ts
├── package.json
├── tsconfig.json
├── .env.example
└── ARCHITECTURE.md
```

6. Technology Stack

Layer	Technology	Purpose
Smart Contracts	Solidity ^0.8.20	On-chain logic
Contract Toolchain	Hardhat + Foundry	Compile, test, deploy
Contract Libraries	OpenZeppelin Contracts v5	RBAC, access control, standards
ZK Circuits	Circom 2.x	Circuit DSL for ownership proof
ZK Prover/Verifier	SnarkJS	Proof generation & Solidity verifier export
Proving Scheme	Groth16	Succinct, non-interactive ZK proofs
Hash Function	Poseidon	ZK-friendly hash for Merkle trees & nullifiers
Stealth Crypto	secp256k1 ECDH	Key derivation for stealth addresses
Token Standards	ERC-721, ERC-1155	Supported RWA token types (unmodified)
Frontend Framework	Next.js 14 (App Router) + TypeScript	Web application
Web3 Libraries	ethers.js v6, wagmi v2, RainbowKit	Wallet connection & contract interaction
Blockchain	Ethereum Sepolia, Polygon Mumbai	Testnet deployment targets
Dev Tools	Hardhat, Foundry, Remix IDE, MetaMask	Development & testing
Version Control	Git + GitHub	Source control & open-source release

7. Directory Structure (Target)

Recommended monorepo layout using **npm workspaces** or **turborepo**:

```
null0wn/
├── contracts/      # Solidity (Hardhat project root)
├── circuits/       # Circom circuits + compiled artifacts
├── scripts/        # Deployment & setup scripts
├── test/           # Unit & integration tests
├── frontend/       # Next.js app
├── docs/           # Architecture diagrams, research writeup
├── hardhat.config.ts
├── package.json    # Root workspace package.json
└── ARCHITECTURE.md # This file
```

8. Smart Contract Interfaces

IStealthRegistry (ERC-6538)

```
interface IStealthRegistry {
    /// @notice Register a stealth meta-address for msg.sender
    /// @param stealthMetaAddress Encoded (spendPubKey, viewPubKey)
    function registerKeys(bytes calldata stealthMetaAddress) external;

    /// @notice Retrieve the registered stealth meta-address for a user
    function getStealthMetaAddress(address user) external view returns (bytes memory);

    event StealthMetaAddressSet(address indexed registrant, bytes stealthMetaAddress);
}
```

IStealthAnnouncer (ERC-5564)

```
interface IStealthAnnouncer {
    /// @notice Emit an announcement after a stealth transfer
    /// @param schemeId Stealth address scheme identifier
    /// @param stealthAddress The one-time stealth address used
    /// @param ephemeralPubKey Sender's ephemeral public key R
    /// @param metadata Optional encrypted payment metadata
    function announce(
        uint256 schemeId,
        address stealthAddress,
        bytes calldata ephemeralPubKey,
        bytes calldata metadata
    ) external;

    event Announcement(
        uint256 indexed schemeId,
        address indexed stealthAddress,
        address indexed caller,
        bytes ephemeralPubKey,
        bytes metadata
    );
}
```

IOwnershipVerifier

```
interface IOwnershipVerifier {
    /// @notice Submit a ZK ownership proof for on-chain verification
    /// @param proof Groth16 proof: [a[0], a[1], b[0][0], b[0][1], b[1][0], b[1][1], c[0], c[1]]
    /// @param pubSignals Public signals: [merkleRoot, threshold, nullifier]
    function verifyOwnership(
        uint256[8] calldata proof,
        uint256[3] calldata pubSignals
    ) external returns (bool);

    event VerifiedOwnership(
        bytes32 indexed nullifier,
        bytes32 indexed merkleRoot,
        uint256 timestamp
    );
}
```

ComplianceModule

```
// Uses OpenZeppelin AccessControl
contract ComplianceModule is AccessControl {
    bytes32 public constant REGULATOR_ROLE = keccak256("REGULATOR_ROLE");

    struct AuditRequest {
        address regulator;
        bytes32 scope;           // jurisdiction or asset class identifier
        uint256 timestamp;
        bytes encryptedResponse;
    }

    mapping(uint256 => AuditRequest) public auditLog;
    uint256 public auditCount;

    /// @notice Pre-authorized regulator requests a scoped audit
    function requestAudit(bytes32 scope) external onlyRole(REGULATOR_ROLE);

    /// @notice Admin fulfills the audit with encrypted data
    function fulfillAudit(uint256 auditId, bytes calldata encryptedData)
        external onlyRole(DEFAULT_ADMIN_ROLE);

    event AuditRequested(uint256 indexed auditId, address indexed regulator, bytes32 scope);
    event AuditFulfilled(uint256 indexed auditId, uint256 timestamp);
}
```

9. ZK Circuit Design

ownership_proof.circom (high-level structure)

```
pragma circom 2.0.0;

include "merkle_inclusion.circom";
include "nullifier.circom";
include "poseidon/poseidon.circom";

template OwnershipProof(merkleDepth) {
  // --- Private inputs ---
  signal input tokenId;
  signal input spendKeyHash;
  signal input merklePathElements[merkleDepth];
  signal input merklePathIndices[merkleDepth];

  // --- Public inputs ---
  signal input merkleRoot;
  signal input threshold;
  signal input nullifier;

  // Constraint 1: nullifier = Poseidon(spendKeyHash, tokenId)
  component nullifierCheck = NullifierDerivation();
  nullifierCheck.spendKeyHash <== spendKeyHash;
  nullifierCheck.tokenId <== tokenId;
  nullifierCheck.out == nullifier;

  // Constraint 2: tokenId is in the Merkle tree
  component merkleCheck = MerkleInclusion(merkleDepth);
  merkleCheck.leaf <== tokenId;
  merkleCheck.root <== merkleRoot;
  for (var i = 0; i < merkleDepth; i++) {
    merkleCheck.pathElements[i] <== merklePathElements[i];
    merkleCheck.pathIndices[i] <== merklePathIndices[i];
  }

  // Constraint 3: ownership >= threshold (enforced via range check)
  // ... implementation depends on data model
}

component main {public [merkleRoot, threshold, nullifier]} = OwnershipProof(20);
```

Trusted Setup

- Use **Groth16** over **BN254** curve (supported by Ethereum precompile `ecPairing` / EIP-197)
- Use existing Powers of Tau (e.g., Hermez `powersOfTau28_hez_final_18.ptau`) for testnet
- Export Solidity verifier: `snarkjs zkey export solidityverifier circuit_final.zkey`
`Groth16Verifier.sol`

Circuit Artifacts (commit to repo under `circuits/build/`)

- `ownership_proof.r1cs`
- `ownership_proof.wasm` — loaded by browser for witness generation
- `ownership_proof_final.zkey` — proving key
- `verification_key.json` — used by SnarkJS off-chain verifier
- `Groth16Verifier.sol` — auto-generated, never hand-edit

10. Frontend Architecture

Stack

- **Next.js 14** with App Router and TypeScript
- **wagmi v2** + **RainbowKit** for wallet connection
- **ethers.js v6** for contract interaction
- **SnarkJS** (WASM build) for client-side proof generation

Pages & Routes

Route	Purpose
/	Landing page / dashboard — wallet status, stealth key status
/stealth	Generate/import stealth keys, scan announcements, view privately received assets
/prove	Generate ZK ownership proof client-side, submit to verifier contract
/compliance	Export compliance attestation for regulators

Key Client-Side Library Responsibilities

lib/stealth.ts

- Generate spend key pair and view key pair (secp256k1)
- Encode stealth meta-address from spend + view public keys
- Derive one-time stealth address from recipient's meta-address + ephemeral key (ECDH)
- Scan ERC-5564 `Announcement` events to discover tokens sent to user
- Derive private spend key for a given stealth address

lib/zkProver.ts

- Load circuit WASM and zkey from `/public/circuits/`
- Build witness from user inputs (tokenId, spendKeyHash, merklePath)
- Call `snarkjs.groth16.fullProve()`
- Format proof for Solidity verifier calldata (`uint256[8]` , `uint256[3]`)
- Return `{ proof, publicSignals }`

lib/merkle.ts

- Build Poseidon-hashed Merkle tree from list of token IDs
- Compute inclusion proof (path elements + left/right indices) for a given leaf
- Expose `merkleRoot` as public signal for ZK proof

lib/contracts.ts

- Export typed contract instances (using ethers.js + ABI)
- Read addresses from `NEXT_PUBLIC_*` environment variables

Non-Custodial Guarantee

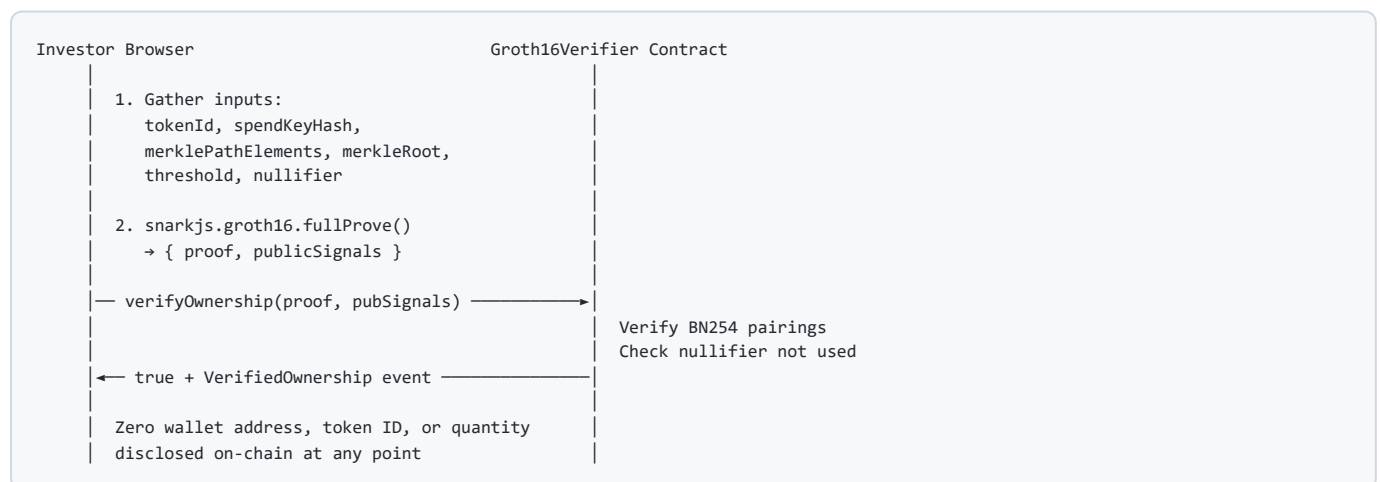
- Private keys never leave the browser — all signing and ZK proving runs client-side
- No backend server stores sensitive user data
- Stealth keys may be stored in browser `localStorage` (AES-encrypted with a user password)

11. Data Flow Diagrams

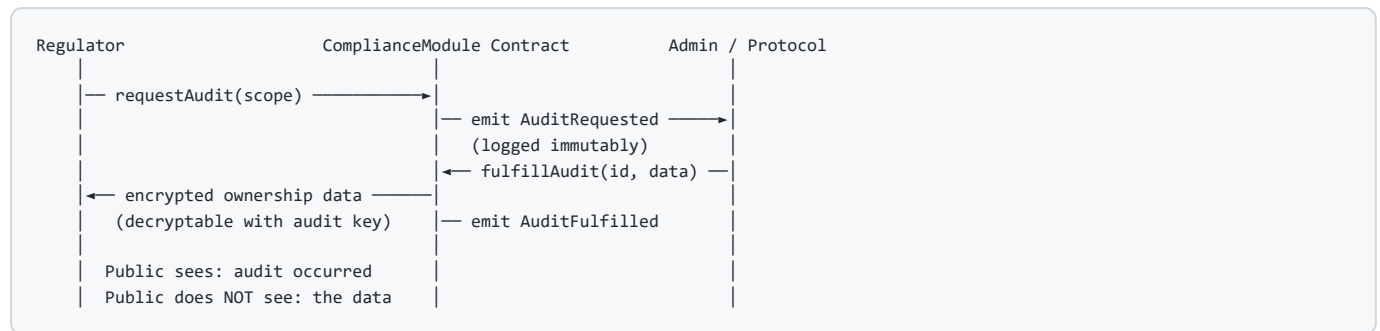
Stealth Transfer Flow



ZK Proof Submission Flow



Compliance Audit Flow



12. Key Cryptographic Primitives

Primitive	Usage	Library
secp256k1 ECDH	Stealth address derivation (shared secret computation)	@noble/curves
Poseidon Hash	ZK-friendly Merkle tree nodes and nullifier derivation	circomlibjs
Merkle Tree	Ownership inclusion proofs (depth ~20 for 1M leaves)	Custom, using Poseidon
Groth16	ZK proof generation and on-chain verification	SnarkJS + Circom
BN254 (alt_bn128)	Elliptic curve for Groth16 proofs; native Ethereum precompile	EIP-197
keccak256	Standard Solidity hashing (events, mappings)	Solidity built-in
AES-GCM	Optional client-side encryption of stored stealth keys	Web Crypto API

13. Scope & Constraints

In Scope

- Stealth address generation, registration, and scanning (ERC-5564 / ERC-6538) on Ethereum Sepolia / Polygon Mumbai
- ZK proof circuit design and compilation in Circom (Groth16, ownership attestation)
- On-chain Groth16 verifier smart contract deployment and integration
- Compliance audit module with RBAC for regulatory key holders
- Compatibility with ERC-721 and ERC-1155 RWA tokens (**no token contract modifications**)
- Non-custodial web frontend (Next.js, TypeScript, ethers.js, wagmi)
- Open-source release on GitHub

Out of Scope

- Cross-chain RWA asset bridging (future work)
- Non-EVM blockchain support
- Issuance or tokenization of new RWAs (NullOwn is a privacy layer only)
- Mobile application
- Mainnet deployment (testnet only for this phase)

Critical Constraints for AI-Assisted Development

These are hard rules that must never be violated:

1. **Do not modify RWA token contracts** — NullOwn is a wrapper/overlay, not a replacement. ERC-721/1155 contracts are treated as black boxes.
2. **Groth16Verifier.sol is auto-generated** — never hand-edit this file; always regenerate via `snarkjs zkey export solidityverifier`.
3. **All ZK proof generation must run client-side** — never send private keys, spend keys, or circuit witnesses to a server.
4. **Nullifiers must be checked for double-use** — the verifier contract must maintain a `mapping(bytes32 => bool) public usedNullifiers` and revert on reuse.
5. **Audit log entries must be immutable** — once written on-chain, audit records cannot be deleted or modified.
6. **No private key storage in plaintext** — if stealth keys are persisted in localStorage, they must be AES-encrypted.

14. Development Conventions

Solidity

- Version: `^0.8.20` with explicit `pragma`
- Use OpenZeppelin Contracts v5 for all access control and standard implementations
- Follow [NatSpec](#) for all public/external functions
- Emit events for every state-changing operation
- Use custom errors (`error Unauthorized()`) rather than string revert messages for gas efficiency
- File naming: `PascalCase.sol` ; one contract per file

TypeScript / Frontend

- `"strict": true` in `tsconfig.json`
- No `any` types — define proper interfaces for all contract return types and circuit signals
- Use `wagmi` hooks for all wallet interactions; never use raw `window.ethereum`
- Keep ZK proving logic in `lib/zkProver.ts` — do not mix business logic with UI components
- `async/await` throughout; no raw Promise chains

Circom Circuits

- Each sub-circuit in its own `.circom` file; compose using `include` and `component`
- Add inline comments explaining each constraint group
- Test circuits independently using SnarkJS input JSON files before integrating with the frontend
- Circuit depth parameter (e.g., Merkle depth) should be a template argument, not hardcoded

Testing

- Smart contracts: Hardhat + Chai + ethers.js; target 100% branch coverage on critical paths
- ZK circuits: Test with known valid witnesses and known invalid witnesses (expect proof failure)
- Frontend: Unit tests for all `lib/` functions using Vitest; do not require a live network for unit tests
- Integration tests: Use Hardhat local node

Environment Variables

```
# .env.example – copy to .env and fill in values; never commit .env

# Deployer
PRIVATE_KEY=                # 0x... deployer wallet private key

# RPC endpoints
SEPOLIA_RPC_URL=
MUMBAI_RPC_URL=

# Block explorer
ETHERSCAN_API_KEY=

# Deployed contract addresses (set after deployment, used by frontend)
NEXT_PUBLIC_ERC5564_ADDRESS=
NEXT_PUBLIC_ERC6538_ADDRESS=
NEXT_PUBLIC_VERIFIER_ADDRESS=
NEXT_PUBLIC_COMPLIANCE_ADDRESS=

# Network
NEXT_PUBLIC_CHAIN_ID=11155111      # 11155111 = Sepolia, 80001 = Mumbai
```

Git Conventions

- Branch naming: `feature/<name>` , `fix/<name>` , `chore/<name>`
- Commit style: Conventional Commits (`feat:` , `fix:` , `chore:` , `docs:`)
- Never commit: `.env` , `node_modules/` , `circuits/build/*.zkey` (too large — use Git LFS or document download steps)
- Always commit: `circuits/build/*.wasm` , `circuits/build/verification_key.json` , `contracts/zk/Groth16Verifier.sol`

15. References

#	Title	URL
1	ERC-5564: Stealth Address Protocol	https://eips.ethereum.org/EIPS/eip-5564
2	ERC-6538: Stealth Meta-Address Registry	https://eips.ethereum.org/EIPS/eip-6538
3	Circom Documentation	https://docs.circom.io
4	SnarkJS Library	https://github.com/iden3/snarkjs
5	OpenZeppelin Contracts	https://docs.openzeppelin.com/contracts
6	Hardhat Development Environment	https://hardhat.org/docs
7	Groth16 Proving Scheme (original paper)	https://eprint.iacr.org/2016/260.pdf
8	Poseidon Hash Function	https://eprint.iacr.org/2019/458.pdf
9	BN254 / alt_bn128 Ethereum Precompile	https://eips.ethereum.org/EIPS/eip-197
10	circomlibjs (Poseidon JS)	https://github.com/iden3/circomlibjs
11	@noble/curves (secp256k1)	https://github.com/paulmillr/noble-curves

This document should be updated whenever major architectural decisions change. Keep it as the single source of truth for system design. Last updated: 2026-08-04.